

Computer Programming Summer Interest Group

Session 16 / Abstraction

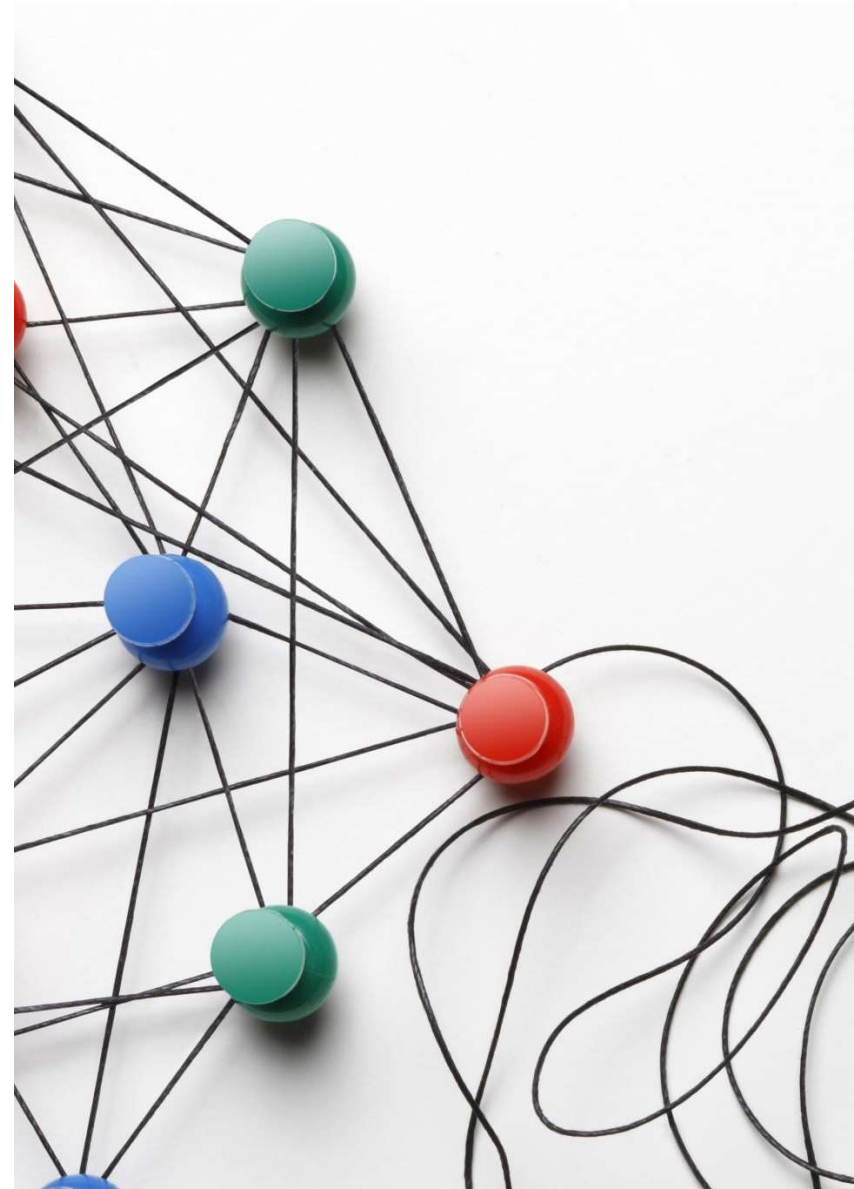
1 John 4:7-21

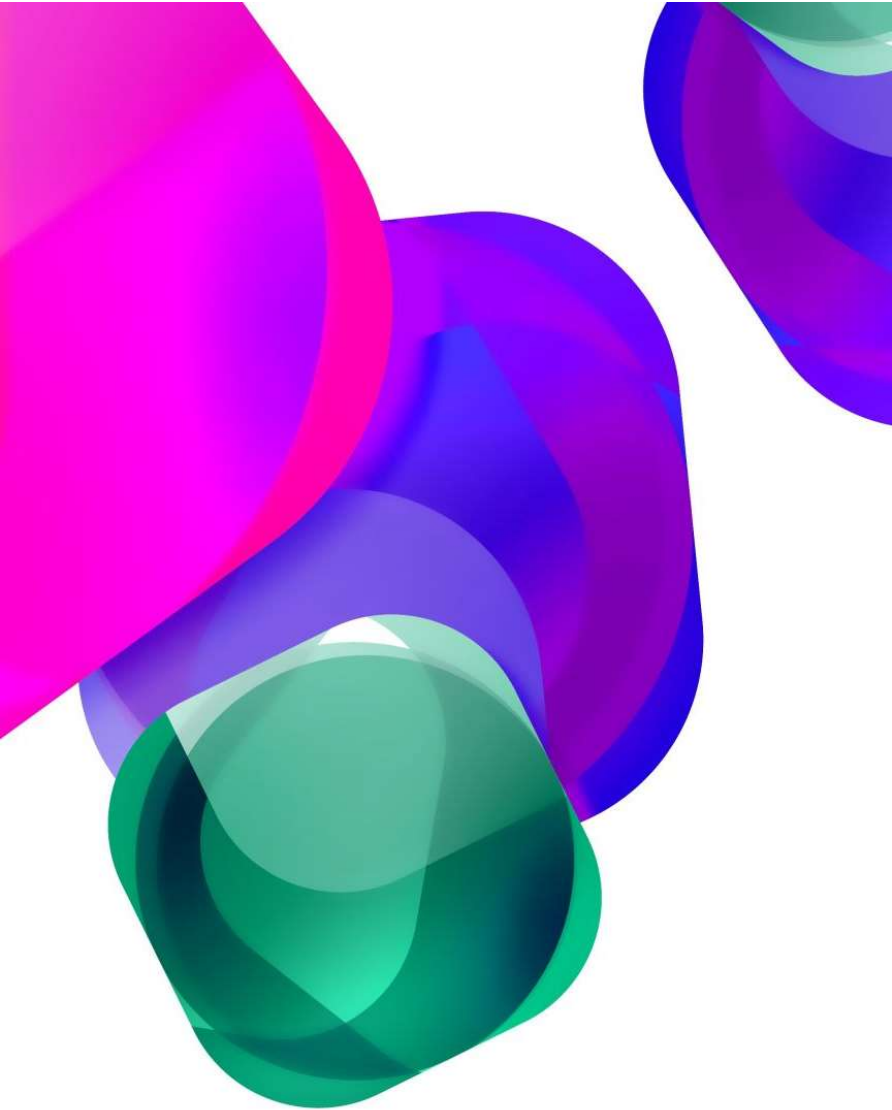
Beloved, let us love one another, for love is from God, and whoever loves has been born of God and knows God. Anyone who does not love does not know God, because God is love. In this the love of God was made manifest among us, that God sent his only Son into the world, so that we might live through him. In this is love, not that we have loved God but that he loved us and sent his Son to be the propitiation for our sins. Beloved, if God so loved us, we also ought to love one another. No one has ever seen God; if we love one another, God abides in us and his love is perfected in us.

By this we know that we abide in him and he in us, because he has given us of his Spirit. And we have seen and testify that the Father has sent his Son to be the Savior of the world. Whoever confesses that Jesus is the Son of God, God abides in him, and he in God. So we have come to know and to believe the love that God has for us. God is love, and whoever abides in love abides in God, and God abides in him. By this is love perfected with us, so that we may have confidence for the day of judgment, because as he is so also are we in this world. There is no fear in love, but perfect love casts out fear. For fear has to do with punishment, and whoever fears has not been perfected in love. We love because he first loved us. If anyone says, "I love God," and hates his brother, he is a liar; for he who does not love his brother whom he has seen cannot love God whom he has not seen. And this commandment we have from him: whoever loves God must also love his brother.

Objectives

- By the end of this session, you will be able to:
 - Understand abstract classes using `abc.ABC`
 - Create abstract methods with `@abstractmethod`
 - Define and use protocol classes with `typing.Protocol`
 - Compare and contrast abstract classes and protocol classes





Abstraction

- Abstraction is a concept in programming which exposes a capability without exposing the underlying implementation.
 - Hiding implementation details allows the exposed interface to be tested to ensure the implementation functions as intended
 - This allows the implementation to be changed while ensuring functionality is preserved.
- Python was not designed to hide the implementation
 - All Python scripts have full exposure, and developers can examine them at will.
 - But in general, trying to leverage undocumented behavior is a great way to break a program when the implementation changes.

Abstraction and Inheritance

- Content in the previous session noted that inheritance could be misused.
 - Abstraction can be used in conjunction with inheritance to make it safer.
 - Python provides two approaches to support abstraction
 - **Abstract Base Classes** – Allow partial implementation of a class with the remaining elements required to conform to a pre-defined interface
 - **Protocols** – Define an interface with no implementation details, but which specify how conforming classes must be presented to protocol users.
-



Abstract Base Classes

- An abstract base class allows partial implementation of a class but reserves some methods (called abstract methods) to be specified in child classes
- No child class can be instantiated unless all abstract methods are defined
 - It will raise `TypeError` if such is attempted
- We use inheritance to create a strong link from the base class to all of the child classes
 - This can still lead to violations of the Open / Closed principle when not used with care

Abstract Base Class Sample



Programs using Abstract Base Classes must import the ABC class and the `abstractmethod` decorator from the `abc` package



Each of the classes can have additional attributes and methods (not shown here)



`Agnostic` cannot be instantiated because it does not define a `faith` method



`Christian` implements both `love` and `faith` so it can be instantiated.



Decorate abstract methods with `@abstractmethod` in the abstract base class and leave them undefined



Implement abstract methods in child classes

```
from abc import ABC, abstractmethod
```

```
class Faithful(ABC):
```

```
    @abstractmethod
```

```
    def love(self):
```

```
        pass
```

```
    @abstractmethod
```

```
    def faith(self) -> float:
```

```
        pass
```

```
class Agnostic(Faithful):
```

```
    def love(self):
```

```
        print("love for world")
```

```
class Christian(Faithful):
```

```
    def love(self):
```

```
        print("love for God")
```

```
    def faith(self):
```

```
        print("has faith")
```

```
def express_faith(
    person: Faithful):
    print(type(person))
    person.love()
    person.fait
```

```
alice = Christian()
express_faith(person=alice)
```

```
# Cannot instantiate an Agnostic
# since it is defined without a
# faith method. When run, this
# will generate a TypeError
# exception:
```

```
bob = Agnostic()
express_faith(person=bob)
```

Protocol Classes

- A protocol specified an interface only.
 - A protocol class should not have any attributes or any fully defined functions
- There is no inheritance between a protocol specification and a class which implements the protocol
 - Python uses “duck-typing” (If it looks like a duck, swims like a duck and quacks like a duck, it’s probably a duck) to determine adherence to the protocol
- This approach is generally thought to be more “Pythonic” than Abstract Base Classes
 - Easier to implement, more flexible, reduced “boiler-plate”



Protocol sample

- Import “Protocol” class from “typing” package.
- Inherit from Protocol class to define protocol interface
- Implementing classes adhere to the Protocol class definition
- No checks to assess conformance
- Attempts to access unimplemented methods will generate an “AttributeError” exception
- No direct relation between the Protocol class and classes implementing it (i.e. no inheritance)

```
from typing import Protocol

class Faithful(Protocol):
    def love(self):
        pass

    def faith(self) -> float:
        pass

class Agnostic:
    def love(self):
        print("love of world")

    def skepticism(self):
        print("has skepticism")

class Christian:
    def love(self):
        print("love of God")

    def faith(self):
        """Express faith"""
        print("has faith")

def express_faith(
    person: Faithful):
    print(type(person))
    person.love()
    person.faitth()

alice = Christian()
express_faith(person=alice)

bob = Agnostic()
express_faith(person=bob)
```

Abstract Base Classes vs. Protocols

Abstract Base Classes

- Provides a direct linkage to the parent class
- Can bind other methods and attributes to child classes derived from the base class
- Has additional syntactic boilerplate

Protocols

- Simpler to implement
- Reduces coupling since no direct linkage is made between classes
- Better aligns with Python approach to use “duck-typing”



Activities

1. Rewrite the Shape, Circle and Rectangle classes from the previous session activity to use Abstract Base Classes.
 - Write a function which takes as an argument an instance of Shape and prints the area and perimeter of the Shape instance
2. Rewrite the Shape, Circle and Rectangle classes from the previous session activity to use a Protocol instead of and Abstract Base Class.
 - You should be able to reuse the function used to print the area and perimeter from the previous activity.
3. In both cases, consider using exception handling to gracefully reject arguments passed to the function that do not implement the needed methods.